

■ ARRAY ROTATION — COMPLETE NOTES

■ 1. THEORY

What is Array Rotation?

Array rotation means shifting elements of an array circularly by k positions.

Right Rotation

$[1, 2, 3, 4, 5]$, $k = 2 \rightarrow [4, 5, 1, 2, 3]$

Left Rotation

$[1, 2, 3, 4, 5]$, $k = 2 \rightarrow [3, 4, 5, 1, 2]$

■ Circular Nature (VERY IMPORTANT)

Arrays behave like a circle in rotation.

$\text{new_index} = (i + k) \% n$ (right rotation)

$\text{new_index} = (i - k + n) \% n$ (left rotation)

■ Key Observations

$k = k \% n$

Rotation $\neq$ shifting $\rightarrow$ It is repositioning of elements

■ 2. ROTATION BY 1

Using Extra Array

```
int arr[] = {3, 4, 5, 6};
int n = 4;

int temp[4];

temp[0] = arr[n-1];

for(int i = 1; i < n; i++){
    temp[i] = arr[i-1];
}
```

■ Intuition:

Build a new array with correct positions

In-Place Method

```

int last = arr[n-1];

for(int i = n-1; i >= 1; i--){
    arr[i] = arr[i-1];
}

arr[0] = last;

```

■ Important Concept:

Shift from BACK to avoid overwriting values

```

WRONG:
arr = [1,2,3,4]

i = 0 → arr[1] = arr[0]
i = 1 → arr[2] = arr[1] ■ data lost

CORRECT:
i = 3 → arr[3] = arr[2]
i = 2 → arr[2] = arr[1]
i = 1 → arr[1] = arr[0]

```

■ 3. ROTATION BY K

Extra Array Method

```
temp[(i + k) % n] = arr[i];
```

■ Intuition:

Directly place each element at its final index

In-Place Brute Force

```

for(int j = 0; j < k; j++){
    int last = arr[n-1];

    for(int i = n-1; i >= 1; i--){
        arr[i] = arr[i-1];
    }

    arr[0] = last;
}

```

Time Complexity: $O(n * k)$

■ 4. REVERSAL ALGORITHM (OPTIMAL)

```

void reverseArray(int arr[], int start, int end){
    while(start < end){
        swap(arr[start], arr[end]);
        start++;
        end--;
    }
}

```

■ Intuition:

Reverse breaks structure, partial reverse rebuilds correct order

Original: [A B C D]

$k = 2$

```
Reverse all:    [D C B A]
Reverse first k: [C D B A]
Reverse rest:   [C D A B]
```

Time: $O(n)$, Space: $O(1)$ → BEST

■ 5. LEFT ROTATION (REVERSAL)

```
reverseArray(arr, 0, k-1);
reverseArray(arr, k, n-1);
reverseArray(arr, 0, n-1);
```

■ 6. EDGE CASES

$k = 0$, $k > n$, $n = 1$, empty array, negative k

■ FINAL SUMMARY

Extra Array → $O(n)$, $O(n)$

Brute Force → $O(n*k)$, $O(1)$

Reversal → $O(n)$, $O(1)$ BEST ■

■ FINAL INTERVIEW INSIGHT

Rotation problems are about index mapping, segment manipulation, and circular behavior